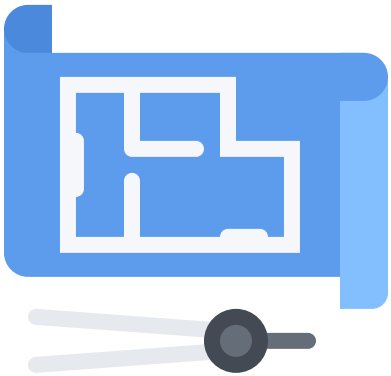
Heated Plate Project

Dr. Kyle Horne

ME-3640 – Heat Transfer

2026-Spring



Contents

1 Deliverables

☑ Result 1 Read Mesh

Write code which completes the following tasks:
(a) Read mesh
(b) Plot problem (See Figure 4)

☑ Result 2 Build Matrices

Write code which completes the following tasks:
(a) Compute conduction matrix $[\mathrm{K}]$
(b) Compute capacitance matrix $[\mathrm{C}]$
(c) Plot conduction matrix sparsity (See Figure 5)

☑ Result 3 Compute Steady-state

Write code which completes the following tasks:
(a) Apply steady boundary conditions
(b) Setup linear system $[\mathrm{A}]\{\mathrm{T}\} = \{\mathrm{b}\}$
(c) Solve linear system for temperature
(d) Plot solution (See Figure 6)

☑ Result 4 Compute Transient

Write code which completes the following tasks:
(a) Apply transient boundary conditions
(b) Setup linear system $[\mathrm{A}]\{\mathrm{T}\}^{new} = [\mathrm{B}]\{\mathrm{T}\}^{\mathrm{old}} + \{\mathrm{b}\}$
(c) Solve linear system for each step
(d) Plot solution time evolution (See Figure 7)

☑ Result 5 Quality Code

- All functions need docstrings
  - Document all inputs and outputs for the function
  - Explain the function's purpose
  - Connect appropriate functions to the math being done
- LLM assistance is permitted!
  - Document which tool was used in comments
  - Identify what problem the tool was used to solve
  - Clearly explain how the generated code works in your own words

☑ Result 6 Learning Narrative

Write a short (no more than two pages) narrative which discusses your experience writing this solver. Your narrative must address at least the following topics. Additional topics may be included at your discretion.

- Identify which aspects were the easiest to understand, and those which were the hardest. Clearly indicate what cause the ease or difficulty.
- Describe your debugging process. What strategies worked, and what didn't? Did you use AI/LLMs? If yes, what was the experience like?
- Explain any changes in your attitudes towards solvers in commercial engineering software (ANSYS, SolidWorks, *etc.*) after writing your own solver.
- What recommendations would you make to improve this project for next semester?

2 Introduction

Originally proposed by Patankar in his seminal work on the simulation of fluid dynamics, the dual-mesh approach to unstructured conservation law problems combines the best aspects of finite-volume and finite element methods. This project will require students to master the concept of the dual-mesh and the practical aspects of unstructured solver operation. These methods (or very nearly related ones) are used in all commercial engineering analysis tools for computational heat transfer (CHX) and fluid flow (CFD).

The core idea of dual-mesh solvers is to use shape functions defined across each element (triangles in our case) to evaluated the terms needed to enforce conservation laws over cells/control-volumes built out of portions of these elements. Since the elements and cells overlap in a way the places the cell boundaries strictly inside the elements, this permits simple evaluation of the shape functions to compute the needed cell face terms. Heat conduction requires knowledge of the temperature gradient in order to compute the conductive fluxes because of Fourier's law.

$$\dot{\vec{q}} = -k\nabla T \tag{1}$$

Conveniently, a triangle's three nodes can be used to uniquely identify a plane with a uniform gradient. This means that the gradient over each element may be assumed to be uniform, making application of Fourier's law much simpler.

3 Conservation Law

$$\{\dot{Q}\}_{in} - \{\dot{Q}\}_{out} + \dot{E}_{gen} = \dot{E}_{store} \tag{2}$$

The single defining characteristic of the Control Volume Finite Element Method (CVFEM) is the use of a dual-mesh comprising both elements for solution interpolation and cells for the application of conservation laws. Figure 1 shows an example of the dual mesh with a small number of nodes, elements, and cells. Note how the same set of nodes are connected by elements by contained by cells. This makes the elements suitable for defining field variations across the domain while the cells are better-suited for articulating conservation laws around each node. As is typical for control volume (Finite Volume) methods, nodes lay at the centers of each cell.

These two complementary discretizations of the problem domain play distinct roles in the constructing the linear system of equations used to approximate the target partial differential

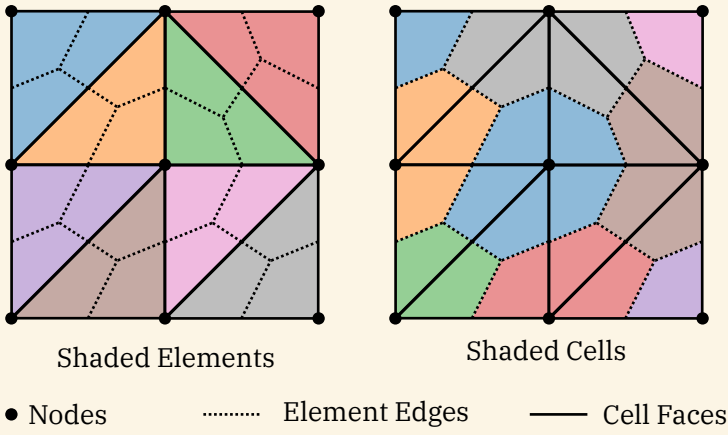


Figure 1: Examples of a dual-mesh visualized by shading the elements (left) and cells (right). Element and cell boundaries are also indicated using solid or dashed lines.

equation. In our case, the target equation is the heat conduction equation but these same methods have been successfully applied to problems in fluid dynamics, solid mechanics, and many other fields.

Elements are typically defined by a mesh generation software, and the cells are then constructed dynamically from these during the solution process. Somewhat atypically, solvers using the CVFEM iterate over elements as they construct residual expressions for the cells partially comprised of those elements. This process stands in contrast with solvers written to operate on regular grids, which typically iterate over the cells (control volumes) directly.

3.1 Element Coordinates & Definition

Each element in the mesh is a triangle consisting of three nodes (one at each corner). The nodes for a single triangular element are depicted in Figure 2.

These node coordinates may be packed as rows of a matrix as seen in Equation 3. In this arrangement, $\mathbf{x}$ coordinates are in the first column while $\mathbf{y}$ coordinates are found in the second.

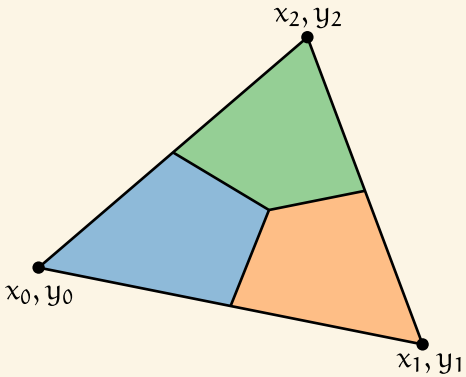


Figure 2: Anatomy of a single element in a dual-mesh arrangement. Each triangular element includes fractions of three different cells. Nodes at the triangle's corners form the basis of the cell definitions. Cell faces are formed by connecting each element (triangle) edge to the element's centroid.

$$[x] = \begin{bmatrix} x_0 & y_0 \\ x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} \tag{3}$$

When combined with a column of 1′s these may be the position used in a polynomial representation of temperature variation through the element as seen in Equation 4 and Equation 5.

$$[X]\{c\} = \{T\} \tag{4}$$

$$\begin{bmatrix} 1 & x_0 & y_0 \\ 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \end{bmatrix} \begin{Bmatrix} c_0 \\ c_1 \\ c_2 \end{Bmatrix} = \begin{Bmatrix} T_0 \\ T_1 \\ T_2 \end{Bmatrix} \tag{5}$$

The vector $\{c\}$ constitutes the coefficients of a linear 2D polynomial (a plane) which takes the specified temperature values $\{T\}$ values at the triangle's corners. If the nodal temperatures $\{T\}$ are know, the matrix $[X]$ may be inverted to compute this polynomial's coefficients $\{c\}$.

$$\{c\} = [X]^{-1}\{T\} \tag{6}$$

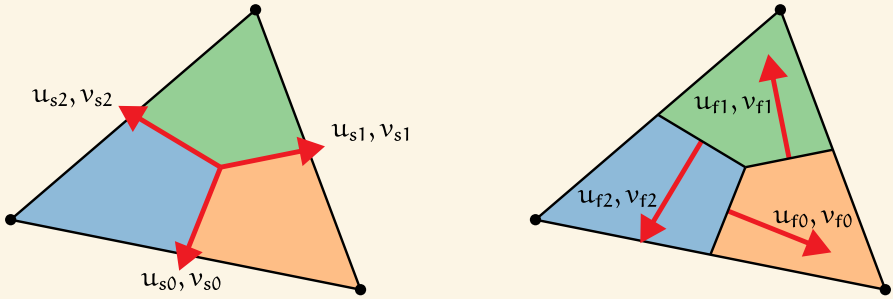


Figure 3: Vector definitions for the cell boundaries contained within each element.

Left Side vectors from the element centroid to each side's midpoint. These vectors may be readily computed from the element's coordinates x_i, y_i.

Right Face vectors from each cell to the others within an element. Rotating the cell face vectors by a quarter turn makes vectors normal to each side. By rotating without scaling, these vectors' magnitudes match the side length. This makes them suitable for computing heat rates from heat fluxes.

3.2 Gradient Extraction

Careful selection of the linear coefficients from $\{c\}$ permits construction of the gradient vector for this element, which is uniform across the element since the plane spanning the triangle only has a single gradient. This selection may be accomplished through multiplication by the matrix $[S]$, which looks mostly like an offset version of the identity matrix and works in similar manner. The defintion of $[S]$ can be found in Equation 8.

$$\{g\} = [S]\{c\} = [S][X]^{-1}\{T\} \tag{7}$$

$$\begin{Bmatrix} c_1 \\ c_2 \end{Bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{Bmatrix} c_0 \\ c_1 \\ c_2 \end{Bmatrix} \tag{8}$$

3.3 Cell Side Definitions

With each element's gradient now well definied, attention must be directed towards computing the surface area vectors for each cell's boundaries within the element. Cell boundaries in the dual mesh are defined by joining the mid-points of each triangle edge to the mid-point of the triangles themselves. Figure 3 depicts this arrangement for an example triangle. The three cells intersecting this triangle are shaded by color to distinguish them, while vectors are shown pointing from the triangular element's mid-point to the mid-point of each element edge.

Computing these vectors (identified as x_{si}) can be accomplished through matrix multiplication and subtraction. The matrix [E] is defined so that [E][x] produces the edge mid-points as rows of a matrix. The matrix [M] computes the element's mid-point in duplicate for all three rows as [M][x]. The difference of these products produces the desired side vectors packed into a matrix as rows, as seen in Equation 9.

$$[u_s] = ([E] - [M])[x]$$
$$\begin{bmatrix} u_{s0} & v_{s0} \\ u_{s1} & v_{s1} \\ u_{s2} & v_{s2} \end{bmatrix} = \left(\frac{1}{2} \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \right) \begin{bmatrix} x_0 & y_0 \\ x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} \tag{9}$$

While these vectors are not directly needed for the solver, they are required to compute the cell face vectors which directly contribute to calculation of the cell face fluxes.

3.4 Cell Face Definitions

The side length of each cell face is well defined by the side vectors computed in Section 3.3. Unfortunately, these vectors point from the element mid-point to the element edge mid-points, which is not useful to compute cell face fluxes as needed by the solver. This can be resolved by (right) multiplying each vector by a rotation matrix which has the effect of rotating each vector by a quarter-turn in the anti-clockwise direction. Equation 10 shows this process symbolically, and

Equation 11 expands all defined terms back to the elements corner node coordinates. A graphical depiction of these vectors in relation to their parent element and owner cells is found in Figure 3.

$$[u_f] = [u_s][R]\ \Delta z = ([E] - [M])[x][R]\ \Delta z \tag{10}$$

$$\begin{bmatrix} u_{f0} & v_{f0} \\ u_{f1} & v_{f1} \\ u_{f2} & v_{f2} \end{bmatrix} = \Delta z \begin{bmatrix} u_{s0} & v_{s0} \\ u_{s1} & v_{s1} \\ u_{s2} & v_{s2} \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \tag{11}$$

With both the gradient vector and cell face area vectors defined, the total heat rate across each cell face may be computed and used to build a solver.

3.5 Cell Face Heat Rate

The total heat rate passing through each cell face may be computed by approximating the integral of heat conduction over that surface (Equation 12).

$$\dot{Q}_{face} = \int_{\partial V} \vec{q} \cdot d\vec{A} \approx \left(\vec{\dot{q}} \cdot \vec{u}_f \right)_{midpoint} \tag{12}$$

Using this logic, a vector of the three cell face heat rates defined for a particular element may be computed by multiplying the face area vector matrix against the gradient and thermal conductivity. This may be written as in Equation 13 and Equation 14.

$$\{f\} = -k[u_f]\{g\} \tag{13}$$

$$\begin{Bmatrix} f_0 \\ f_1 \\ f_2 \end{Bmatrix} = -k\ \Delta z \begin{bmatrix} u_{f0} & v_{f0} \\ u_{f1} & v_{f1} \\ u_{f2} & v_{f2} \end{bmatrix} \begin{Bmatrix} c_1 \\ c_2 \end{Bmatrix} \tag{14}$$

With the cell face heat rates computed, the net contribution for each of the three cells partially constructed from a particular element may be computed. By looping over all elements and accumulating the net contribution to each cell from each element, the total net heat rate for each cell can be computed.

3.6 Conduction Matrix

The finite volume method requires computing the net heat rate into each cell in the mesh. In the case of the dual mesh arrangement used here, this means that each element can only compute a partial contribution of heat rate to each cell. The vector of partial heat rates for each cell partially composed of parts of this element is $\partial\{\dot{Q}\}_e$, and must be computed using integration of the flux over each cell's surface as in Equation 15. In this expression, the ∂ symbol is used to designate that this heat rate only includes surfaces fluxes from an incomplete portion of the total cell surface.

$$\{\dot{Q}\}_{in} - \{\dot{Q}\}_{out} = -\int_{\partial V} \dot{\vec{q}} \cdot d\vec{A} \tag{15}$$

This may be expressed using matrix arithmetic for the cell boundaries contained within each element. Pre-multiplication of the element heat rate vector $\{f\}$ by a difference matrix $[D]$ computes the net heat rate into each cell from surfaces in this element.

$$\begin{aligned} \partial\{\dot{Q}\}_e &= -\int_{\partial V,e} \dot{\vec{q}} \cdot d\vec{A} \approx -[D]\{f\} \\ [D] &= \begin{bmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} \end{aligned} \tag{16}$$

Careful substitution of expressions from the prior sections permits the matrix product $[D]\{f\}$ to be expressed in terms of nodal temperatures and geometric factors. Careful attention to the signs from Equation 16 and Equation 14 show an overall positive sign for the righ-hand-side of Equation 17

$$-[D]\{f\} = k\ \Delta z\ [D]([E] - [M])[x][R][S][X]^{-1}\{T\} \tag{17}$$

Since all terms on the right-hand-side of this equation are know except the temperature, they can all be conveniently grouped into an element conduction matrix.

$$\begin{aligned} \partial\{\dot{Q}\}_e &= [K]_e\{T\} \\ [K]_e &= k\ \Delta z\ [D]([E] - [M])[x][R][S][X]^{-1} \end{aligned} \tag{18}$$

Assembly of the global conduction matrix $[K]$ simply requires the accumulation of the conduction matrix from each element. Summation of all elemental contributions to the global matrix yields the overall conduction matrix.

$$[K] = \sum_{\leftrightarrow} \left([K]_e\right) \tag{19}$$

3.7 Capacity Matrix

Each cell's energy storage can be represented by a thermal capacitance, per the usual definition found in Equation 20.

$$\dot{E}_{store} = \int_V \rho \cdot C_P \dot{T}\, dV \tag{20}$$

Since these are to be computed on a per element basis, the capacitance of each cell's portion of the element can be computed based on the volume of that cell in that element. Using a simple mid-point approximation, the needed integral can be approximated directly through multiplication by each cell's partial volume.

$$\int_V \rho \cdot C_P \dot{T}\, dV \approx \rho \cdot C_P \lceil V \rfloor \{\dot{T}\} = [C]_e \{\dot{T}\} \tag{21}$$

Finally, the elemental capacity matrix can be computed as a diagonal matrix of these capacitance values.

$$\begin{aligned} [C]_e &= \frac{\Delta z}{2} \cdot \rho \cdot C_P \cdot |X| \lceil I \rfloor \\ \lceil V \rfloor &= \frac{\Delta z}{2 \cdot 3} \cdot |X| \lceil I \rfloor \\ |X| &= \begin{vmatrix} 1 & x_0 & y_0 \\ 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \end{vmatrix} \end{aligned} \tag{22}$$

This result relies on a useful property of the determinant operator $|?|$, which can compute twice the area of a triangle when the coordinates are appropriately arranged. Summation of all elemental contributions to the global matrix yields the overall capacitance matrix.

$$[C] = \sum_{\leftrightarrow} \left([C]_e \right) \tag{23}$$

4 Solution Methods

With all the needed matrices and vectors defined, actual solvers for the steady-state and transient cases may be constructed. These sections detail how these are arranged from the mathematical tools previously defined.

4.1 Steady-state Problem Assembly

Solution of the steady-state heat transfer for this problem can be accomplished by first writing the steady-state conservation law.

$$\{\dot{Q}\}_{in} - \{\dot{Q}\}_{out} + \dot{E}_{gen} = 0 \tag{24}$$

The various terms in this conservation law must them be approximated by matrix operators on the solution temperature vector.

$$\{\dot{Q}\}_{in} - \{\dot{Q}\}_{out} \approx [K]\{T\} \tag{25}$$

$$\dot{E}_{gen} \approx \{E\} \tag{26}$$

The resulting equation can then be re-arranged into a format suitable for solution using the regular techniques of linear algebra.

$$[K]\{T\} = -\{E\} \tag{27}$$

For reference, this linear system will be discussed elsewhere in the standard linear algebra format, with the exception that the solution vector is represented by {T}.

$$[A]\{T\} = \{b\} \tag{28}$$

4.2 Transient Problem Assembly

The transient version of this problem can be written using the usual conservation law format as below:

$$\dot{E}_{store} = \{\dot{Q}\}_{in} - \{\dot{Q}\}_{out} + \dot{E}_{gen} \tag{29}$$

The spatial operators are approximated by the matrices [C] and [K], as well as the vector {E}. It should be noted that the transient terms have been left alone.

$$[C]\{\dot{T}\} = [K]\{T\} + \{E\} \tag{30}$$

Both sides are integrated over a single time step. Without any approximation in treatment of the transient term, the left side is simplified through the fundamental theorem of calculus.

$$\int_{\Delta t} [C]\{\dot{T}\}\ dt = \int_{\Delta t} \Big([K]\{T\} + \{E\} \Big)\ dt \tag{31}$$

$$[C]\Big(\{T\}^{new} - \{T\}^{old} \Big) = [K] \int_{\Delta t} \{T\}\ dt + \int_{\Delta t} \{E\}\ dt \tag{32}$$

A very simple integration scheme is applied to approximate the integral on the right side of Equation 32. If a single value were to prevail over the time step, that value times the step duration would be a reasonable approximation of the integral.

$$\int_{\Delta t} [K]\{T\}\ dt \approx \{T\}^{?}\ \Delta t \tag{33}$$

The problem lies in which value to assume prevails. A weighted average of the current and next time steps can be defined and adjusted using the β parameter.

$$\{T\}^{?} = \beta\{T\}^{new} + (1-\beta)\{T\}^{old}$$

Substituting this weighted average into the integral in Equation 32 and simplifying yields a useful result.

$$[C]\{T\}^{new} - [C]\{T\}^{old} \approx [K]\Big(\beta\{T\}^{new} + (1-\beta)\{T\}^{old}\Big)\ \Delta t + \{E\}\Delta t \tag{34}$$

Moving all "new" terms to the left and "old" terms to the right, this expression is then factored into a format suitable for solution using linear algebra.

$$[C]\{T\}^{new} - \beta[K]\{T\}^{new}\ \Delta t \approx (1-\beta)[K]\{T\}^{old}\ \Delta t + [C]\{T\}^{old} + \{E\}\Delta t \tag{35}$$

$$([C] - \beta[K]\Delta t)\{T\}^{new} \approx ([C] + (1-\beta)[K]\Delta t)\{T\}^{old} + \{E\}\Delta t \tag{36}$$

Once again these terms are then put into a generalized format for discussion elsewhere.

$$\begin{aligned} &[A]\{T\}^{new} = [B]\{T\}^{old} + \{b\} \\ &[A] = [C] - \beta[K]\Delta t \qquad [B] = [C] + (1-\beta)[K]\Delta t \qquad \{b\} = \{E\}\Delta t \end{aligned} \tag{37}$$

5 Boundary Conditions

Correctly treating heat transfer within a domain is only half the battle when writing a simulation such as this. Boundary conditions must be handled correctly for any solution to properly represent actual physical systems.

5.1 Insulated Boundary (Adiabatic)

When a boundary is defined as adiabatic, that requires that there be no heat transfer through that surface. The finite-volume method must represent all heat transfer on all cell surfaces through terms. If no term for a surface is defined, this surface is then implicitly adiabatic. This is called the *natural* boundary condition, because it is what happens without intervention.

5.2 Fixed Flux ↔ Generation Vector

Heat is added to cells in both the steady and transient cases. This permits the same logic to be used when computing the source vector $\{E\}$ in both cases.

The total heat rate into a cell from a fixed flux boundary condition is the integral of the flux over the cell's surface.

$$\dot{E} = \int_{l} \Delta z \ \dot{q} \ dl \tag{38}$$

For each boundary element, its length can be computed directly using the familiar Euclidean distance formula. Half this length can be used to compute the partial cell surface exposed to this condition. An elemental heat rate vector can then be computed as $\{E\}_e$.

$$\{E\}_e \approx \Delta z \ \dot{Q} \frac{l}{2} \begin{Bmatrix} 1 \\ 1 \end{Bmatrix} \tag{39}$$

These elemental heat rate vectors must then be accumulated across all elements in the mesh. Summation of all elemental contributions to the global vector yields the overall generation vector.

$$\{E\} = \sum_{\leftrightarrow} \left(\{E\}_e \right) \tag{40}$$

5.3 Fixed Temperature

Asserting a know temperature on boundaries within the mesh plays out differently in the steady and transient cases.

Steady

In the case of steady state solutions, the discretized system of equations may be written as follows:

$$[A]\{T\} = \{b\} \tag{41}$$

To force a particular temperature in this format to assume a known value, the required modifications to the system are relatively straight-forward. All coefficients in the row of $[A]$ which corresponds to the target temperature must first be zeroed out. Then, the diagonal term should be assigned to the value one. Lastly, the right-hand side vector $\{b\}$ should be set to the required temperature value for this row of the system.

In terms of the conduction and generation equations these adjustments can be made nearly without change. Changes intended for [A] should be directed to [K], and changes to {b} should be applied oppositely to {E}.

$$[K]\{T\} = -\{E\} \tag{42}$$

Transient

Transient methods generally require solution of linear system of equations at each time step, as seen below:

$$[A]\{T\}^{new} = [B]\{T\}^{old} + \{b\} \tag{43}$$

If a temperature boundary condition is know and does not change through time, a good solution is to set the initial temperature correctly then ensure it does not change. This could be accomplished through adjustments to [A], [B], and {b}. The relevant rows in [A] need to be zeroed and then one assigned to the diagonal terms. Matrix [B] requires the same treatment. The vector {b} simply would need to be zeroed out for these rows.

As in the steady case, the needed modifications can be accomplished through careful adjustments to the physical matrices. All terms in [K] for the impacted rows should be set to zero. The same should be done for {E} Since this leaves the values of [C] isolated on the left and right sides, no adjustments are needed to the capacitance matrix.

$$[A] = [C] - \beta[K]\Delta t \qquad [B] = [C] + (1-\beta)[K]\Delta t \qquad \{b\} = \{E\}\Delta t \tag{44}$$

These changes effectively thermally isolate the fixed temperatures from all other nodes, causing their values to hold regardless of other temperatures.

A Rubrics

Submissions of this assignment will be graded upon the criteria found in this section. Review these carefully, and complete your work so that application of this rubric produces your desired score.

Criterion 1 Read Mesh

Code correctly reads and plots mesh.

Points	Description
10	Mesh is correctly read and plotted.
0	Feature does not work at all OR is plagiarized OR generated by AI without notification.

Criterion 2 Build Matrices

Code correctly builds matrix sparsity patterns.

Points	Description
10	Matrix is constructed in a way the creates the appropriate sparsity pattern.
5	Matrix is successfully constructed, but demonstrates incorrect sparsity pattern.
0	Feature does not work at all OR is plagiarized OR generated by AI without notification.

Criterion 3 Compute Steady

Code correctly computes steady-state solution.

Points	Description
20	Computed steady-state temperature is correct.
10	Steady-state solution is successfully computed, but provides incorrect results.
0	Feature does not work at all OR is plagiarized OR generated by AI without notification.

Criterion 4 Compute Transient

Code correctly computes time evolution of center temperature.

Points	Description
20	Computed time evolution of center temperature is correct.
10	Time evolution is successfully computed, but provides incorrect results.
0	Feature does not work at all OR is plagiarized OR generated by AI without notification.

Criterion 5 Quality Code

Code includes needed docstrings, comments, and generally follows best practices.

Points	Description
20	Code is of good general quality.
10	Code is missing any docstrings or required comments.
0	Feature does not work at all OR is plagiarized OR generated by AI without notification.

Criterion 6 Learning Narrative

Narrative includes all required topics and honestly communicates the student experience.

Points	Description
20	Writing covers all required topics and follows standard English language conventions.
10	Writing is missing ANY required topic or includes language so poor it is difficult to understand.
0	Narrative is entirely off topic OR absent OR plagiarized OR generated by AI.

B Mesh File Definitions

Solvers of this kind typically read specialized file formats. These files are created by mesh generation software, which takes geometry definitions from solid models as input. The geometry is then broken into small chunks (normally called elements) and saved as a mesh. This process is shared by finite element and finite volume solvers.

In this project, specialized file formats will not be used. The geometry for this problem has already been processed by the Gmsh tool. Appropriate triangular elements were generated to span the two-dimensional domain. Line elements were generated to represent all physical boundaries of the geometry. In combination, these definitions give the solver the information it needs to construct linear systems of equations which approximate the heat equation.

Mesh information in this project is stored in a mulit-sheet spreadsheet file. Each sheet defines different aspects of the mesh, and are organized into the following:

XY Node positions
IE Internal elements
BE Boundary elements
IC Internal conditions
BC Boundary conditions

Each sheet defines various aspects of the mesh data. Columns in the geometry data sheets are defined in Table 1. Columns in the conditions data sheets are defined in Table 2.

Table 1: Definitions for all columns contained in the mesh file's nodes and elements sheets. Nodes are defined in the sheet `XY`, while sheets `IE` and `BE` define the elements.

Sheet	Column	Data-type	Description
`XY`	Define node positions.		
	`id`	$\mathbb{Z} \geqslant 0$	Node index
	`x`	$\mathbb{R}$	Node x-coordinate in m
	`y`	$\mathbb{R}$	Node y-coordinate in m
`IE`	Define interior (triangular) elements.		
	`id`	$\mathbb{Z} \geqslant 0$	Internal element index
	`cid`	$\mathbb{Z} \geqslant 0$	Internal condition index
	`n0`	$\mathbb{Z} \geqslant 0$	Node-0 index
	`n1`	$\mathbb{Z} \geqslant 0$	Node-1 index
	`n2`	$\mathbb{Z} \geqslant 0$	Node-2 index
`BE`	Define boundary (line) elements.		
	`id`	$\mathbb{Z} \geqslant 0$	Boundary element index
	`cid`	$\mathbb{Z} \geqslant 0$	Boundary condition index
	`n0`	$\mathbb{Z} \geqslant 0$	Node-0 index
	`n1`	$\mathbb{Z} \geqslant 0$	Node-1 index

Table 2: Definitions for all columns contained in the mesh file's internal and boundary conditions sheets. Internal and boundary conditions are defined in sheets `IC` and `BC` respectively.

Sheet	Column	Data-type	Description
`IC`	Define interior conditions.		
	`id`	$\mathbb{Z} \geqslant 0$	Internal condition index
	`name`	`string`	Condition name
	`color`	`string`	Matplotlib-compatible color name
	`k`	$\mathbb{R}$`>0`	Thermal conductivity in $W/m \cdot K$
	`ρ`	$\mathbb{R}$`>0`	Density in kg/m^3
	C_p	$\mathbb{R}$`>0`	Specific Heat $J/kg \cdot K$
	`e`	$\mathbb{R}$`≥0`	Heat generation in W/m^3
`BC`	Define boundary conditions.		
	`id`	$\mathbb{Z} \geqslant 0$	Internal condition index
	`name`	`string`	Condition name
	`color`	`string`	Matplotlib-compatible color name
	`T`	$\mathbb{R}$	Known boundary temperature in °C or K
	`q`	$\mathbb{R}$	Know flux in W/m^2

C Example Solutions

To help students as they work towards functioning solvers, example solutions to this problem are provided. These should be considered as checkpoints as work progresses towards completion. Once the mesh file has been correctly read and plotted, the results should look somewhat like Figure 4. The steady-state case should result in a temperature field looking like the one depicted in Figure 6. Temperature at the center of the domain as it changes through time is plotted in Figure 7.

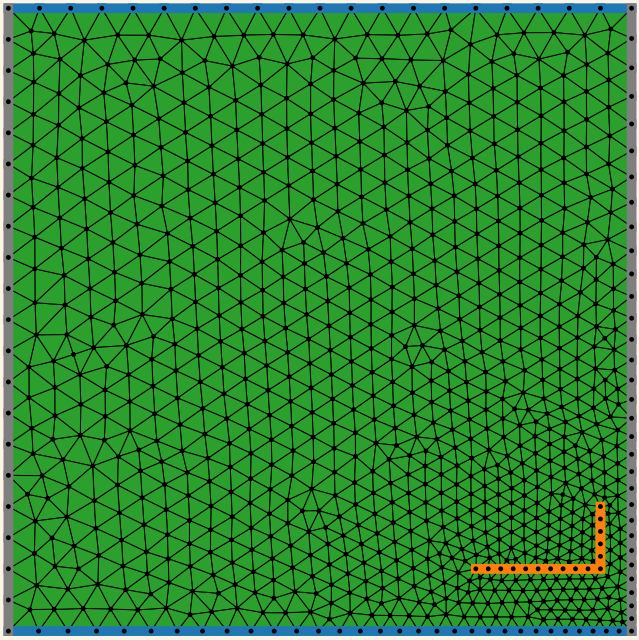


Figure 4: Graphical depiction of the mesh used to approximate solutions to the present problem. Triangular elements were used to fill the two-dimensional space of the domain, with line elements used for the embedded heater and boundaries. Axes have been omitted to simplify the depiction. The line heating element can be seen in the lower right corner, depicted using orange lines for each line element. The north and south boundaries are both shown using blue line elements at the top and bottom of the graphic. The east and west boundaries are shown using gray line elements on the left and right sides of the graphic.

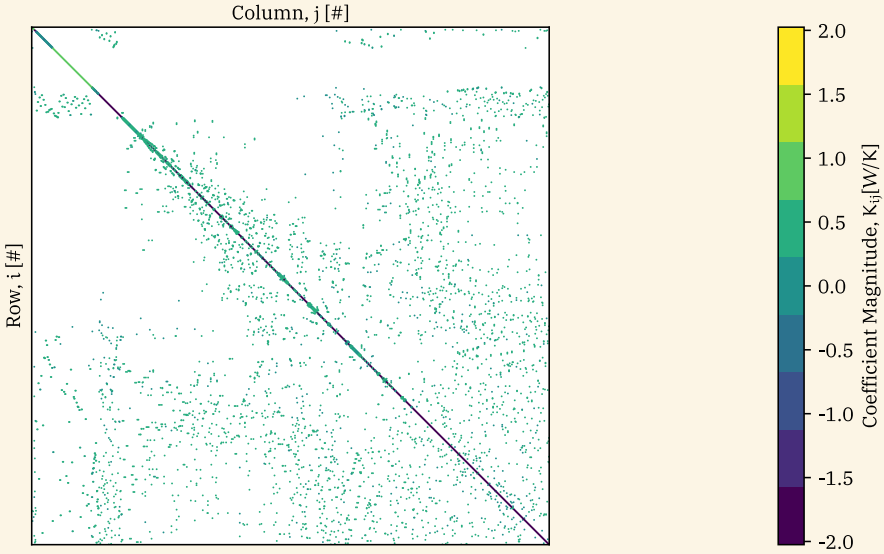


Figure 5: Sparsity pattern for conduction matrix [K]. Strong negative values on the main diagonal are expected from finite-volume methods, and observed here. Locations of non-zero terms in other columns strongly depend on the exact node ordering. A common practice in the past was to reorder nodes in the mesh generation stage to improve the performance of linear system solvers. This is not done here mostly to simplify the process and emphasize the heat transfer aspects of the work.

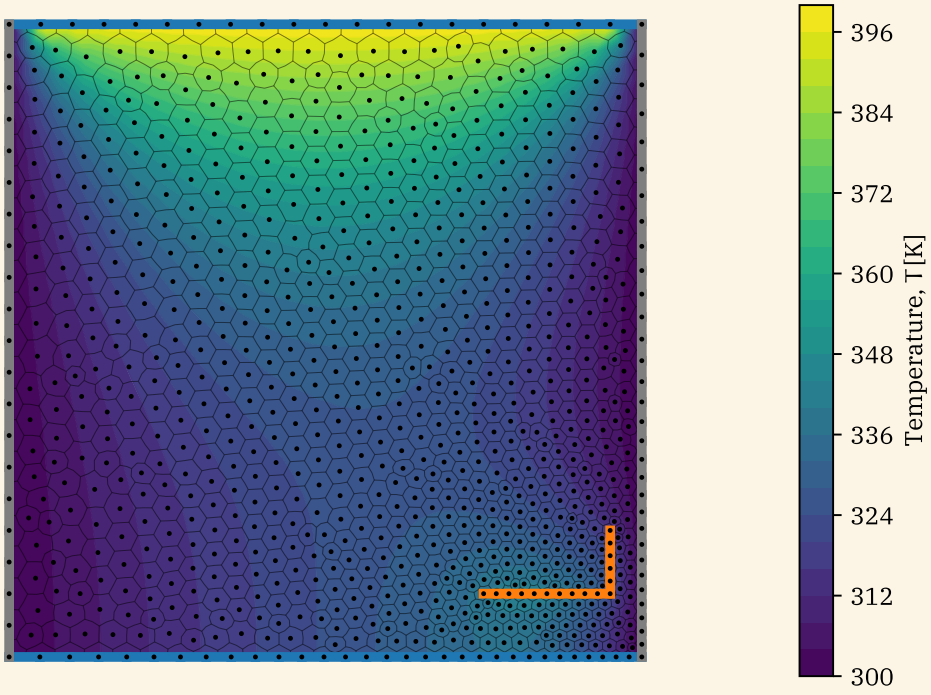


Figure 6: Graphical depiction of the stead-state temperature field for the present problem. Polyhedral cells were used to assert conservation of energy using the finite-volume method. Axes have been omitted to simplify the depiction. Line elements are used to indicate boundaries as in Figure 4 Filled contours indicate the temperature distribution over space. Temperature gradients (and therefore conductive fluxes) are normal to the contour lines.

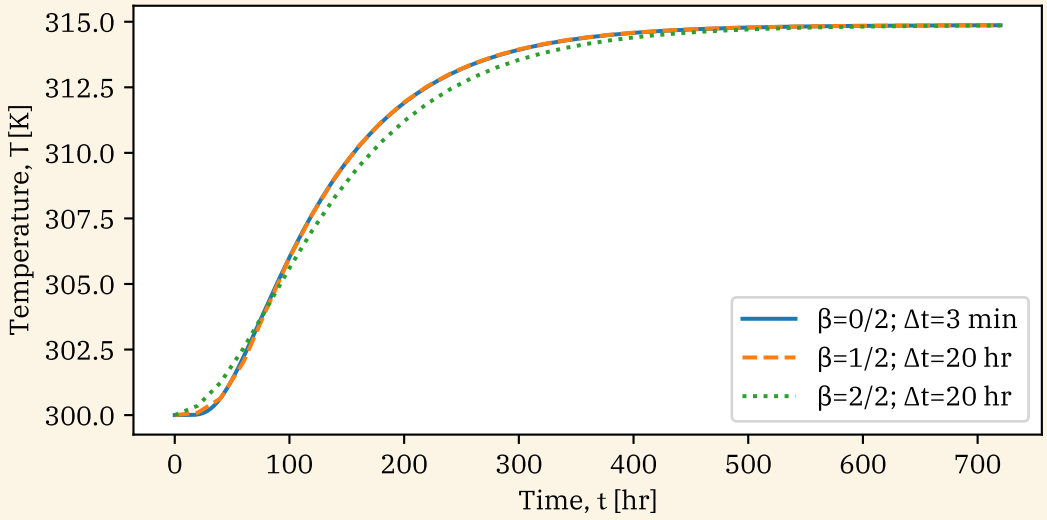


Figure 7: Time evolution of temperature at the point closest to the center of the domain. An initial plateau ends when heat propagation from the line heater and upper boundary makes it to the center. The remaining behavior looks qualitatively like a first-order system response: like Newton's law of cooling. It should be noted that the complexity of the system means a lumped capacitance model would not be suitable in this case. Qualitative understanding of heat transfer, however, remains useful when interpreting complex simulation results.

Each line on the plot shows the performance of a different treatment for the transient term. When $\beta = 0$ the solver is explicit and requires a very small time step for stable results. This has the advantage (dubious...) of ensuring a very accurate solution at the cost of tremendous computational power. Both cases when $\beta = \frac{1}{2}$ (semi-implicit) and $\beta = 1$ (fully-implicit) remain stable with a step size hundreds of times larger. The improved accuracy of the semi-implicit method is clearly seen, as even with a very large step size it achieves the same results as the explicit method.